

Locating and Editing Factual Associations in GPT

Kevin Meng*
MIT CSAIL

David Bau*
Northeastern University

Alex Andonian
MIT CSAIL

Yonatan Belinkov†
Technion – IIT

Abstract

We analyze the storage and recall of factual associations in autoregressive transformer language models, finding evidence that these associations correspond to localized, directly-editable computations. We first develop a causal intervention for identifying neuron *activations* that are decisive in a model’s factual predictions. This reveals a distinct set of steps in middle-layer feed-forward modules that mediate factual predictions while processing subject tokens. To test our hypothesis that these computations correspond to factual association recall, we modify feed-forward *weights* to update specific factual associations using Rank-One Model Editing (ROME). We find that ROME is effective on a standard zero-shot relation extraction (zsRE) model-editing task. We also evaluate ROME on a new dataset of difficult counterfactual assertions, on which it simultaneously maintains both specificity and generalization, whereas other methods sacrifice one or another. Our results confirm an important role for mid-layer feed-forward modules in storing factual associations and suggest that direct manipulation of computational mechanisms may be a feasible approach for model editing. The code, dataset, visualizations, and an interactive demo notebook are available at <https://rome.baulab.info/>.

1 Introduction

Where does a large language model store its facts? In this paper, we report evidence that factual associations in GPT correspond to a localized computation that can be directly edited.

Large language models can predict factual statements about the world (Petroni et al., 2019; Jiang et al., 2020; Roberts et al., 2020). For example, given the prefix “*The Space Needle is located in the city of,*” GPT will reliably predict the true answer: “*Seattle*” (Figure 1a). Factual knowledge has been observed to emerge in both autoregressive GPT models (Radford et al., 2019; Brown et al., 2020) and masked BERT models (Devlin et al., 2019).

In this paper, we investigate how such factual associations are stored within GPT-like autoregressive transformer models. Although many of the largest neural networks in use today are autoregressive, the way that they store knowledge remains under-explored. Some research has been done for masked models (Petroni et al., 2019; Jiang et al., 2020; Elazar et al., 2021a; Geva et al., 2021; Dai et al., 2022; De Cao et al., 2021), but GPT has architectural differences such as unidirectional attention and generation capabilities that provide an opportunity for new insights.

We use two approaches. First, we trace the causal effects of hidden state activations within GPT using causal mediation analysis (Pearl, 2001; Vig et al., 2020b) to identify the specific modules that mediate recall of a fact about a subject (Figure 1). Our analysis reveals that feedforward MLPs at a range of middle layers are decisive when processing the last token of the subject name (Figures 1b,2b,3).

Second, we test this finding in model weights by introducing a Rank-One Model Editing method (ROME) to alter the parameters that determine a feedforward layer’s behavior at the decisive token.

*Equal contribution. Correspondence to mengk@mit.edu, davidbau@northeastern.edu.

†Supported by the Viterbi Fellowship in the Center for Computer Engineering at the Technion.

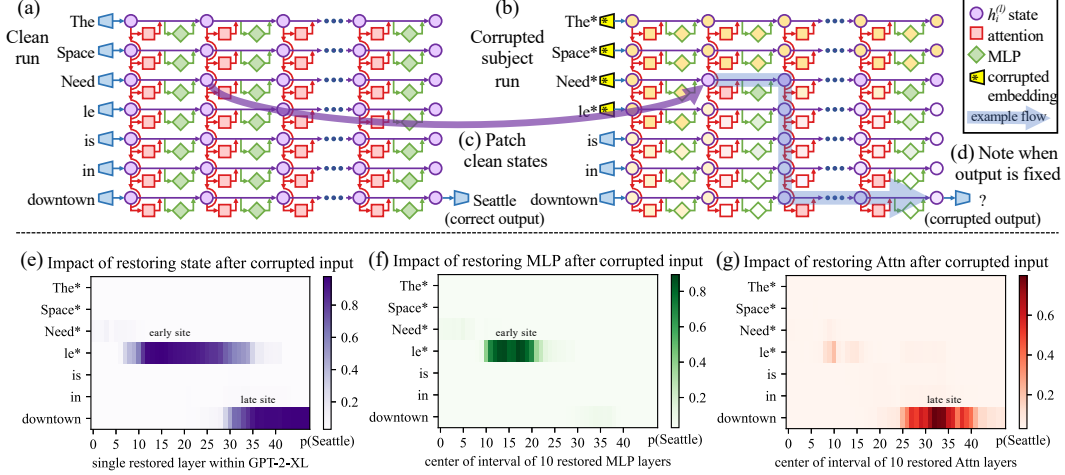


Figure 1: **Causal Traces** compute the causal effect of neuron activations by running the network twice: (a) once normally, and (b) once where we corrupt the subject token and then (c) restore selected internal activations to their clean value. (d) Some sets of activations cause the output to return to the original prediction; the light blue path shows an example of information flow. The causal impact on output probability is mapped for the effect of (e) each hidden state on the prediction, (f) only MLP activations, and (g) only attention activations.

Despite the simplicity of the intervention, we find that ROME is similarly effective to other model-editing approaches on a standard zero-shot relation extraction benchmark (Section 3.2).

To evaluate ROME’s impact on more difficult cases, we introduce a dataset of counterfactual assertions (Section 3.3) that would not have been observed in pretraining. Our evaluations (Section 3.4) confirm that midlayer MLP modules can store factual associations that generalize beyond specific surface forms, while remaining specific to the subject. Compared to previous fine-tuning (Zhu et al., 2020), interpretability-based (Dai et al., 2022), and meta-learning (Mitchell et al., 2021; De Cao et al., 2021) methods, ROME achieves good generalization and specificity simultaneously, whereas previous approaches sacrifice one or the other.

2 Interventions on Activations for Tracing Information Flow

To locate facts within the parameters of a large pretrained autoregressive transformer, we begin by analyzing and identifying the specific hidden states that have the strongest causal effect on predictions of individual facts. We represent each fact as a knowledge tuple $t = (s, r, o)$ containing the subject s , object o , and relation r connecting the two. Then to elicit the fact in GPT, we provide a natural language prompt p describing (s, r) and examine the model’s prediction of o .

An autoregressive transformer language model $G : \mathcal{X} \rightarrow \mathcal{Y}$ over vocabulary V maps a token sequence $x = [x_1, \dots, x_T] \in \mathcal{X}$, $x_i \in V$ to a probability distribution $y \in \mathcal{Y} \subset \mathbb{R}^{|V|}$ that predicts next-token continuations of x . Within the transformer, the i th token is embedded as a series of hidden state vectors $h_i^{(l)}$, beginning with $h_i^{(0)} = \text{emb}(x_i) + \text{pos}(i) \in \mathbb{R}^H$. The final output $y = \text{decode}(h_T^{(L)})$ is read from the last hidden state.

We visualize the internal computation of G as a grid (Figure 1a) of hidden states $h_i^{(l)}$ in which each layer l (left $\rightarrow$ right) adds global attention $a_i^{(l)}$ and local MLP $m_i^{(l)}$ contributions computed from previous layers, and where each token i (top $\rightarrow$ bottom) attends to previous states from other tokens. Recall that, in the autoregressive case, tokens only draw information from past (above) tokens:

$$\begin{aligned} h_i^{(l)} &= h_i^{(l-1)} + a_i^{(l)} + m_i^{(l)} \\ a_i^{(l)} &= \text{attn}^{(l)}(h_1^{(l-1)}, h_2^{(l-1)}, \dots, h_i^{(l-1)}) \\ m_i^{(l)} &= W_{proj}^{(l)} \sigma(W_{fc}^{(l)} \gamma(a_i^{(l)} + h_i^{(l-1)})). \end{aligned} \quad (1)$$

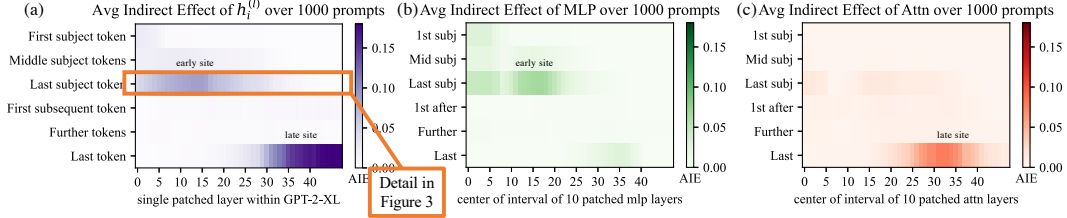


Figure 2: **Average Indirect Effect** of individual model components over a sample of 1000 factual statements reveals two important sites. (a) Strong causality at a ‘late site’ in the last layers at the last token is unsurprising, but strongly causal states at an ‘early site’ in middle layers at the last subject token is a new discovery. (b) MLP contributions dominate the early site. (c) Attention is important at the late site. Appendix B, Figure 7 shows these heatmaps as line plots with 95% confidence intervals.

Each layer’s MLP is a two-layer neural network parameterized by matrices $W_{proj}^{(l)}$ and $W_{fc}^{(l)}$, with rectifying nonlinearity σ and normalizing nonlinearity γ . For further background on transformers, we refer to Vaswani et al. (2017).³

2.1 Causal Tracing of Factual Associations

The grid of states (Figure 1) forms a *causal graph* (Pearl, 2009) describing dependencies between the hidden variables. This graph contains many paths from inputs on the left to the output (next-word prediction) at the lower-right, and we wish to understand if there are specific hidden state variables that are more important than others when recalling a fact.

As Vig et al. (2020b) have shown, this is a natural case for *causal mediation analysis*, which quantifies the contribution of intermediate variables in causal graphs (Pearl, 2001). To calculate each state’s contribution towards a correct factual prediction, we observe all of G ’s internal activations during three runs: a **clean** run that predicts the fact, a **corrupted** run where the prediction is damaged, and a **corrupted-with-restoration** run that tests the ability of a single state to restore the prediction.

- In the **clean run**, we pass a factual prompt x into G and collect all hidden activations $\{h_i^{(l)} \mid i \in [1, T], l \in [1, L]\}$. Figure 1a provides an example illustration with the prompt: “The Space Needle is in downtown _____”, for which the expected completion is o = “Seattle”.
- In the baseline **corrupted run**, the subject is obfuscated from G before the network runs. Concretely, immediately after x is embedded as $[h_1^{(0)}, h_2^{(0)}, \dots, h_T^{(0)}]$, we set $h_i^{(0)} := h_i^{(0)} + \epsilon$ for all indices i that correspond to the subject entity, where $\epsilon \sim \mathcal{N}(0; \nu)$ ⁴. G is then allowed to continue normally, giving us a set of corrupted activations $\{h_{i*}^{(l)} \mid i \in [1, T], l \in [1, L]\}$. Because G loses some information about the subject, it will likely return an incorrect answer (Figure 1b).
- The **corrupted-with-restoration run**, lets G run computations on the noisy embeddings as in the corrupted baseline, *except* at some token i and layer l . There, we hook G so that it is forced to output the clean state $h_i^{(l)}$; future computations execute without further intervention. Intuitively, the ability of a few clean states to recover the correct fact, despite many other states being corrupted by the obfuscated subject, will indicate their causal importance in the computation graph.

Let $\mathbb{P}[o]$, $\mathbb{P}_*[o]$, and $\mathbb{P}_{*, \text{clean } h_i^{(l)}}[o]$ denote the probability of emitting o under the clean, corrupted, and corrupted-with-restoration runs, respectively; dependence on the input x is omitted for notational simplicity. The **total effect** (TE) is the difference between these quantities: $\text{TE} = \mathbb{P}[o] - \mathbb{P}_*[o]$. The **indirect effect** (IE) of a specific mediating state $h_i^{(l)}$ is defined as the difference between the probability of o under the corrupted version and the probability when that state is set to its clean version, while the subject remains corrupted: $\text{IE} = \mathbb{P}_{*, \text{clean } h_i^{(l)}}[o] - \mathbb{P}_*[o]$. Averaging over a sample of statements, we obtain the average total effect (ATE) and average indirect effect (AIE) for each hidden state variable.⁵

³Eqn. 1 calculates attention sequentially after the MLP module as in Brown et al. (2020). Our methods also apply to GPT variants such as Wang & Komatsuzaki (2021) that put attention in parallel to the MLP.

⁴We select ν to be 3 times larger than the empirical standard deviation of embeddings; see Appendix B.1 for details, and see Appendix B.4 for an analysis of other corruption rules.

⁵One could also compute the direct effect, which flows through other model components besides the chosen mediator. However, we found this effect to be noisy and uninformative, in line with results by Vig et al. (2020b).

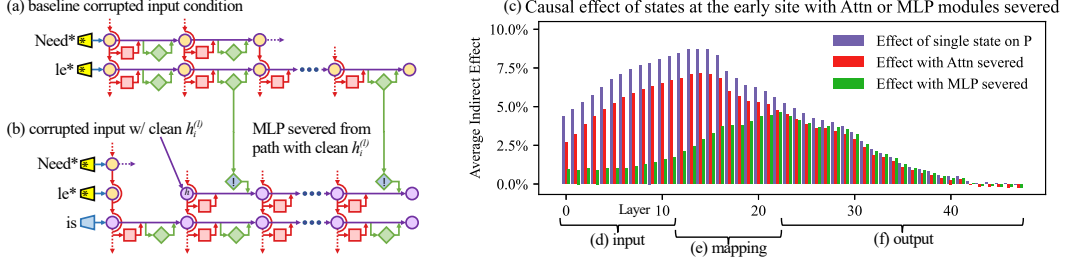


Figure 3: **Causal effects with a modified computation graph.** (a,b) To isolate the effects of MLP modules when measuring causal effects, the computation graph is modified. (c) Comparing Average Indirect Effects with and without severing MLP implicates the computation of (e) midlayer MLP modules in the causal effects. No similar gap is seen when attention is similarly severed.

2.2 Causal Tracing Results

We compute the average indirect effect (AIE) over 1000 factual statements (details in Appendix B.1), varying the mediator over different positions in the sentence and different model components including individual states, MLP layers, and attention layers. Figure 2 plots the AIE of the internal components of GPT-2 XL (1.5B parameters). The ATE of this experiment is 18.6%, and we note that a large portion of the effect is mediated by strongly causal individual states (AIE=8.7% at layer 15) at the last subject token. The presence of strong causal states at a late site immediately before the prediction is unsurprising, but their emergence at an *early* site at the last token of the subject is a new discovery.

Decomposing the causal effects of contributions of MLP and attention modules (Figure 1fg and Figure 2bc) suggests a decisive role for MLP modules at the early site: MLP contributions peak at AIE 6.6%, while attention at the last subject token is only AIE 1.6%; attention is more important at the last token of the prompt. Appendix B.2 further discusses this decomposition.

Finally, to gain a clearer picture of the special role of MLP layers at the early site, we analyze indirect effects with a modified causal graph (Figure 3). (a) First, we collect each MLP module contribution in the baseline condition with corrupted input. (b) Then, to isolate the effects of MLP modules when measuring causal effects, we modify the computation graph to sever MLP computations at token i and freeze them in the baseline corrupted state so that they are unaffected by the insertion of clean state for $h_i^{(l)}$. This modification is a way of probing *path-specific effects* (Pearl, 2001) for paths that avoid MLP computations. (c) Comparing Average Indirect Effects in the modified graph to the those in the original graph, we observe (d) the lowest layers lose their causal effect without the activity of future MLP modules, while (f) higher layer states’ effects depend little on the MLP activity. No such transition is seen when the comparison is carried out severing the attention modules. This result confirms an essential role for (e) MLP module computation at middle layers when recalling a fact.

Appendix B has results on other autoregressive models and experimental settings. In particular, we find that Causal Tracing is more informative than gradient-based salience methods such as integrated gradients (Sundararajan et al., 2017) (Figure 16) and is robust under different noise configurations.

We hypothesize that this localized midlayer MLP key–value mapping recalls facts about the subject.

2.3 The Localized Factual Association Hypothesis

Based on causal traces, we posit a specific mechanism for storage of factual associations: each midlayer MLP module accepts inputs that encode a subject, then produces outputs that recall memorized properties about that subject. Middle layer MLP outputs accumulate information, then the summed information is copied to the last token by attention at high layers.

This hypothesis localizes factual association along three dimensions, placing it (i) in the MLP modules (ii) at specific middle layers (iii) and specifically at the processing of the subject’s last token. It is consistent with the Geva et al. (2021) view that MLP layers store knowledge, and the Elhage et al. (2021) study showing an information-copying role for self-attention. Furthermore, informed by the Zhao et al. (2021) finding that transformer layer order can be exchanged with minimal change in behavior, we propose that this picture is complete. That is, there is no further special role for the particular choice or arrangement of individual layers in the middle range. We conjecture that any fact

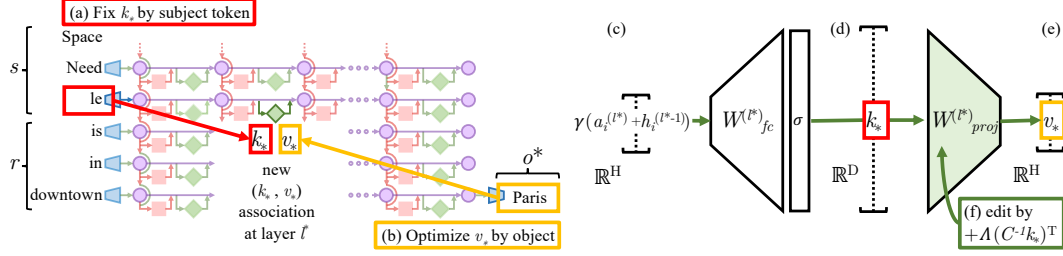


Figure 4: **Editing one MLP layer with ROME.** To associate *Space Needle* with *Paris*, the ROME method inserts a new (k_*, v_*) association into layer l^* , where (a) key k_* is determined by the subject and (b) value v_* is optimized to select the object. (c) Hidden state at layer l^* and token i is expanded to produce (d) the key vector k_* for the subject. (e) To write new value vector v_* into the layer, (f) we calculate a rank-one update $\Lambda(C^{-1}k_*)^T$ to cause $\hat{W}_{proj}k_* = v_*$ while minimizing interference with other memories stored in the layer.

could be equivalently stored in any one of the middle MLP layers. To test our hypothesis, we narrow our attention to a single MLP module at a mid-range layer l^* , and ask whether its weights can be explicitly modified to store an arbitrary fact.

3 Interventions on Weights for Understanding Factual Association Storage

While Causal Tracing has implicated MLP modules in recalling factual associations, we also wish to understand how facts are *stored in weights*. Geva et al. (2021) observed that MLP layers (Figure 4cde) can act as two-layer key-value memories,⁶ where the neurons of the first layer $W_{fc}^{(l)}$ form a *key*, with which the second layer $W_{proj}^{(l)}$ retrieves an associated *value*. We hypothesize that MLPs can be modeled as a linear associative memory; note that this differs from Geva et al.’s per-neuron view.

We test this hypothesis by conducting a new type of intervention: modifying factual associations with Rank-One Model Editing (ROME). Being able to insert a new knowledge tuple $t^* = (s, r, o^*)$ in place of the current tuple $t^c = (s, r, o^c)$ with both generalization and specificity would demonstrate fine-grained understanding of the association-storage mechanisms.

3.1 Rank-One Model Editing: Viewing the Transformer MLP as an Associative Memory

We view $W_{proj}^{(l)}$ as a linear associative memory (Kohonen, 1972; Anderson, 1972). This perspective observes that any linear operation W can operate as a key-value store for a set of vector keys $K = [k_1 \mid k_2 \mid \dots]$ and corresponding vector values $V = [v_1 \mid v_2 \mid \dots]$, by solving $WK \approx V$, whose squared error is minimized using the Moore-Penrose pseudoinverse: $W = VK^+$. Bau et al. (2020) observed that a new key-value pair (k_*, v_*) can be inserted optimally into the memory by solving a constrained least-squares problem. In a convolutional network, Bau et al. solve this using an optimization, but in a fully-connected layer, we can derive a closed form solution:

$$\text{minimize } \|\hat{W}K - V\| \text{ such that } \hat{W}k_* = v_* \text{ by setting } \hat{W} = W + \Lambda(C^{-1}k_*)^T. \quad (2)$$

Here W is the original matrix, $C = KK^T$ is a constant that we pre-cache by estimating the uncentered covariance of k from a sample of Wikipedia text (Appendix E.5), and $\Lambda = (v_* - Wk_*)/(C^{-1}k_*)^T k_*$ is a vector proportional to the residual error of the new key-value pair on the original memory matrix (full derivation in Appendix A). Because of this simple algebraic structure, we can insert any fact directly once (k_*, v_*) is computed. All that remains is to choose the appropriate k_* and v_* .

Step 1: Choosing k_* to Select the Subject. Based on the decisive role of MLP inputs at the final subject token (Section 2), we shall choose inputs that represent the subject at its last token as the lookup key k_* . Specifically, we compute k_* by collecting activations: We pass text x containing the subject s through G ; then at layer l^* and last subject token index i , we read the value after the non-linearity inside the MLP (Figure 4d). Because the state will vary depending on tokens that

⁶Unrelated to keys and values in self-attention.